

修士論文

介護マッチングサービスにおけるマッチング精度向
上の提案と実装

Proposal and Implementation for Improving Matching Accuracy in Elderly Care Matching
Services

北海道大学 大学院情報科学研究院
メディアネットワーク専攻 情報メディア環境学研究室

星子 祐哉

目次

第 1 章	序論	1
第 2 章	研究の目的	12
第 3 章	提案手法	15
第 4 章	実験	17
第 5 章	考察	28
第 6 章	ユーザースタディ	30
第 7 章	回避業務ペナルティと時間志向の実験	32
第 8 章	今後の展望	35
第 9 章	結論	36
システム構成概要（ディレクトリベース）		37
謝辞		38
参考文献		39
参考文献		39
研究業績		40

目次

1.1	介護職員数の推移と必要な増員ペース	2
1.2	介護福祉士資格保有者の内訳	3
1.3	人材確保のための 5 つの視点	4
1.4	介護事業者が直面する課題の構造	5
1.5	北海道における介護人材確保の特殊事情	6
1.6	現行マッチングシステムの定型条件フィルタリング構造	8
1.7	介護特化型マッチングアプリの概要	9
1.8	システムアーキテクチャ概要（主要サービス間の疎結合構成）	10
1.9	ビジネスプランと価値循環（データ駆動の改善サイクル）	11
7.1	20 名データの重み付けパターン比較（概観）	33
7.2	ペナルティ有無 + time_auto の同時評価結果	34

表目次

4.1	予備実験 2 の評価データ（アンカーと類似/非類似職種の例）	22
4.2	BERT モデル別の分離度スコア	22
4.3	本実験 1：ワーカー別の推薦精度	23
4.4	本実験 2 の重み付けパターン一覧	23
4.5	本実験 2（20 件学習）の全パターン	24
4.6	本実験 2（30 件学習）の全パターン	24
4.7	学習データ数による精度推移（Top-5 精度）	25
4.8	特徴量配分パターン（本実験 3）	25
4.9	時間重み付けパターン一覧（1/2）	26
4.10	時間重み付けパターン一覧（2/2）	26
4.11	時間重み付け比較（平均 Top-5）	26
4.12	時間重み付け上位パターン（平均 Top-5）	26
4.13	ステップ別平均 Top-5（有効ワーカー平均）	27
4.14	time_auto 実装結果（5 名, 平均 Top-K）	27
7.1	回避ペナルティ有無の平均精度（18 名, equal / time_auto）	33

概要

近年、日本の介護業界は深刻な人材不足に直面している。厚生労働省の資料によると、2040 年までに約 69 万人の介護人材が不足する見込みである。一方で、有資格でありながら介護職に従事していない「潜在介護士」が約 37 万人存在し、供給可能な人材が活用されていない現状がある。さらに、介護事業者は応募者が集まらない、採用コストの高騰、採用後の定着率の低さといった複合的な課題を抱えており、人材のミスマッチが常態化している。

本研究では、介護特化型マッチングアプリにおけるマッチング精度を向上させるための手法を提案し、実装を行った。現状のマッチングシステムは「勤務地」「日程」「資格要件」などの定型条件マッチが中心であり、希望職種や過去の職務経験と求人内容の意味的な一致は評価されていない。そのため、ワーカーと求人の潜在的な親和性が見逃される可能性が存在する。

提案手法では、自然言語処理モデルを利用した距離計算により、職種の距離（希望職種と実際の職種の一致度）とスキルの距離（職務経歴と求められているスキルの類似度）を測定する。また、時間的距離と勤務時間、過去の応募履歴などの要素を組み合わせる重み付けを行う。実験では、日本語 BERT モデルを含む複数の自然言語処理モデルを横断的に評価し、日本語 Sentence-BERT モデルが最も優れた分離度（0.398）を示すことを確認した。さらに、過去の求人応募データから新規求人との類似度を計算する手法を検証し、BERT による意味的類似検索の有効性を示した。

本研究により、介護マッチングサービスにおいて、ワーカーの希望とより適合した求人を効率的に提示できるシステムの基盤が構築された。今後は、実データを用いた検証と、複数の手法を組み合わせたより高精度なマッチングアルゴリズムの開発、実サービスへの実装とユーザーインタビューを通じた改善サイクルの確立が課題である。

第 1 章

序論

本研究は、介護特化型マッチングサービスにおけるマッチング精度の向上を目的とする。日本の介護業界は深刻な人材不足に直面しており、量的確保と質的マッチングの両面で構造的課題を抱えている。以下、厚生労働省の公式資料に基づき、現状の詳細な分析を行う。

介護人材不足の量的課題

介護人材不足の規模は極めて深刻である。図 1.1 に示すように、2022 年度の介護職員数は約 215 万人であるが、2026 年度には約 240 万人、2040 年度には約 272 万人が必要とされ、現在の増員ペース（年間約 1 万人）では必要な年間 6.3 万人に遠く及ばない状況にある [1]。この人材不足は、単なる労働力の絶対量の問題だけでなく、適切なマッチングが実現できていないという質的課題を同時に抱えている。

■潜在介護福祉士の存在と再就業障壁 介護福祉士国家資格保有者約 47 万人のうち、実際に福祉・介護分野に従事しているのは約 27 万人（約 57%）で、残り約 20 万人（約 43%）が「潜在介護福祉士」として未活用状態にある [2]。図 1.2 に示すように、資格保有者の半数近くが介護職に従事しておらず、この層を活用できれば年間 6.3 万人の 3 年以上の不足をカバーできる可能性がある。

しかし、潜在介護福祉士の再就業を阻む障壁は多岐にわたる。主な要因として、(1) 離職時の職場環境への不満が払拭されない不安、(2) ブランク期間による技術・知識の陳腐化への懸念、(3) 家庭との両立が可能な柔軟な勤務条件を探す困難さ、(4) 求人情報の不透明性（実際の業務内容や職場雰囲気が分からない）が挙げられる。特に、短時間・副業での復帰を希望する層にとって、従来の常勤中心の求人市場では適切な選択肢を見つけにくい。本研究で提案するマッチングシステムは、これらの障壁のうち特に (3) と (4) の解決に焦点を当て、柔軟な働き方と詳細な条件マッチングを実現することで、潜在層の再就業を促進する。

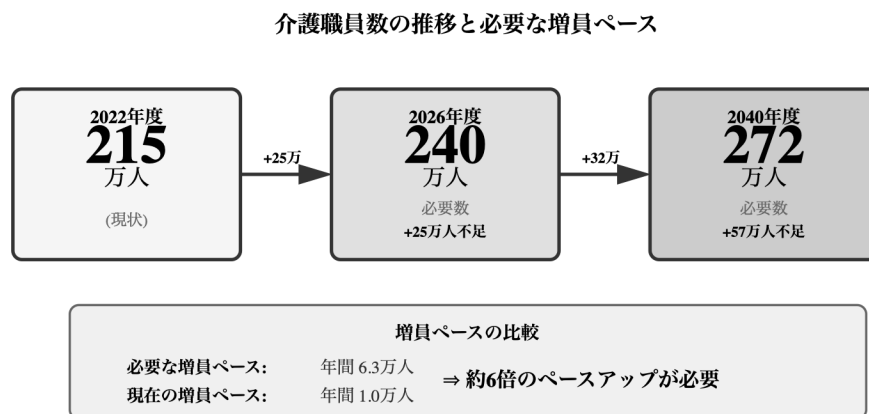


図 1.1: 介護職員数の推移と必要な増員ペース

■**国の施策における位置づけ** 厚生労働省は人材確保のための 5 つの視点の一つとして「潜在的有資格者等の掘り起こし」を明確に位置づけ、再就業促進を政策的に推進している [3]。図 1.3 に示すように、労働環境改善やキャリアパス確立と並んで、潜在介護福祉士の参入促進が重点施策として掲げられている。

さらに、届出制度の整備も「介護福祉士の約 6 割しか介護職に従事しておらず、潜在介護福祉士活用のための方策が必要」との問題認識に基づいている [4]。しかし、制度面の整備だけでは不十分であり、実際に潜在層と事業者を効果的にマッチングする仕組みが求められている。

■**総合的な人材確保戦略** 介護人材確保の総合的取組では、参入促進・資質向上・労働環境改善の 3 つの柱が設定され、未経験者の参入促進と並行して潜在介護福祉士の再就業促進が推進されている [5]。このように、潜在介護福祉士の掘り起こしは国の重点施策とし

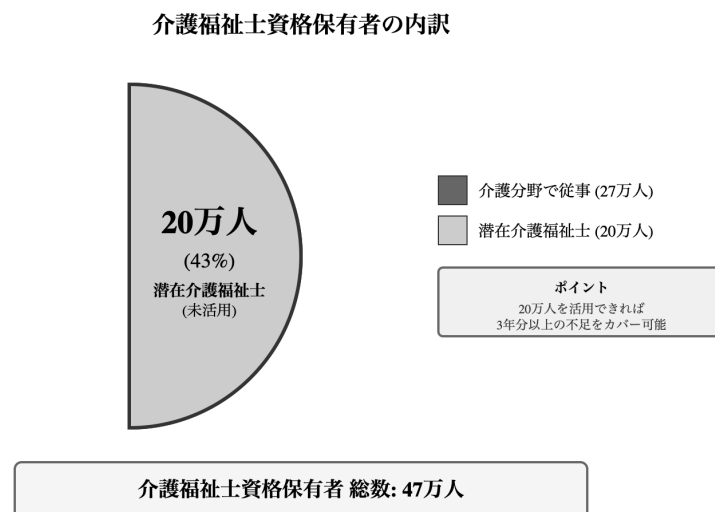


図 1.2: 介護福祉士資格保有者の内訳

て明確に位置づけられており、本研究で提案するマッチングシステムは、この政策目標の実現に直接貢献するものである。

介護業界が直面する課題

介護業界は以下に示す複合的かつ構造的な課題を抱えている。

■人材ミスマッチと経営危機 介護事業者は、応募者が集まらない、採用コストの高騰、採用後の定着率の低さといった複合的な課題に直面している。全国の介護事業所を対象とした調査によれば、約 6 割の事業所が「採用が困難」と回答しており、特に訪問介護事業所では 7 割を超える深刻な状況にある。採用単価は 1 人あたり平均 30～50 万円に達し、中小事業者にとって大きな経営負担となっている。

さらに深刻なのが定着率の低さである。介護職の離職率は約 15% と全産業平均を上回り、採用から 1 年以内の早期離職率は 3～4 割に達するとされる。この背景には、求人票の条件と実際の業務内容の乖離、職場環境への不適応、希望職種とのミスマッチなど、マッ

厚生労働省「介護人材確保のための5つの視点」

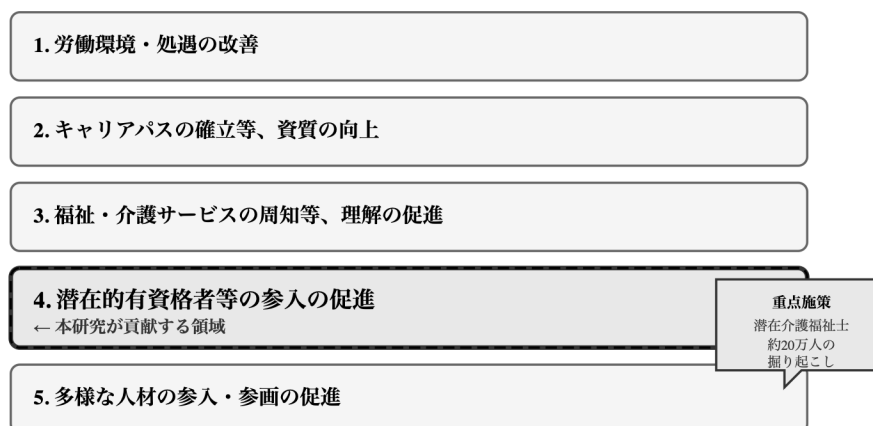


図 1.3: 人材確保のための 5 つの視点

チング精度の問題が潜んでいる。例えば、「身体介護希望」で入職したワーカーが「生活援助中心」の現場に配置されるケースや、「日勤のみ希望」だったにもかかわらず夜勤を求められるケースなど、条件面のミスマッチが離職を誘発している。

人材のミスマッチが常態化した結果、サービスの質低下や廃業リスクの増大という深刻な経営課題を引き起こしている。採用単価 30～50 万円に対して、早期離職率が 3～4 割に達する場合、事業者は採用 1 人あたり実質 90～125 万円のコストを負担することになる（採用費 ×2.5～3 人分）。年間 10 人採用する中小事業者では、年間約 1,000 万円の採用関連コストが発生する計算となり、これは介護報酬の低い業界構造においては極めて重い負担である。マッチング精度が 10 ポイント向上し、早期離職率が 30% に抑制できれば、年間約 200 万円のコスト削減が可能となる。これらの問題は単なる人手不足だけでなく、ワーカーの希望・スキル・経験と求人内容の適切なマッチングが実現できていないことに起因する構造的な課題である。

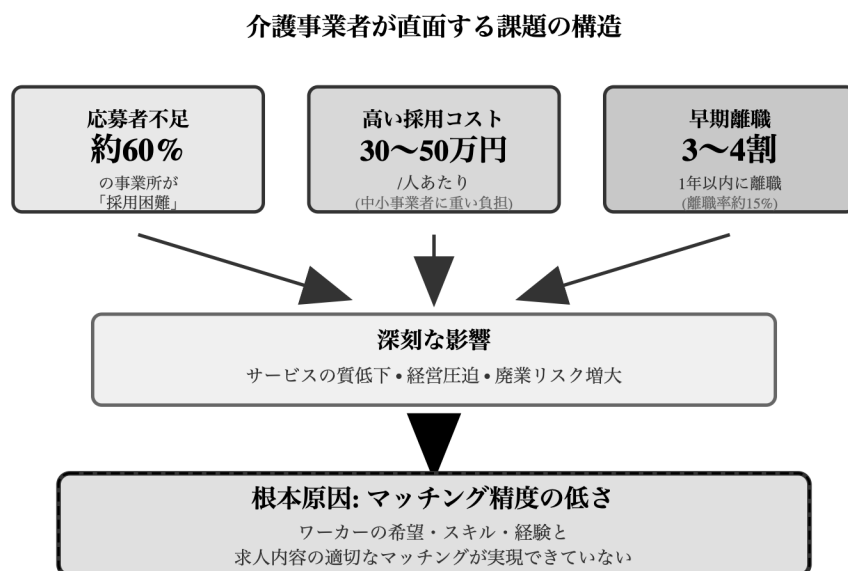


図 1.4: 介護事業者が直面する課題の構造

■**北海道の特殊事情** 本研究の対象地域である北海道は、全国平均と比較して極めて特殊かつ困難な条件下にある。図 1.5 に示すように、高齢化の進行と人口偏在が顕著であり、2025 年には高齢化率が約 33.5% に達すると予想される。これは全国平均（約 30%）を上回る水準である。

人口分布の偏りも深刻で、北海道の総人口約 520 万人のうち、約 3 分の 2 にあたる約 340 万人が札幌市とその周辺地域（石狩振興局管内）に集中している。残りの約 180 万人が、本州の約 5 分の 1 にあたる広大な土地（約 78,000 平方キロメートル）に点在している。この結果、介護人材も札幌圏に集中し、地方部では深刻な人手不足が続いている。

地理的条件による課題も見逃せない。北海道の市町村間の平均距離は本州の 3～4 倍にのぼり、冬季には積雪・凍結により移動困難な期間が年間 4～5 ヶ月続く。例えば、旭川市から道北の稚内市までは約 250km、札幌市から道東の釧路市までは約 300km と、通勤

圏としては現実的ではない距離である。このため、地方部の介護事業所は限られた地域内で人材を確保するしかなく、マッチングの選択肢が極端に狭まっている。

公共交通機関の脆弱さも大きな障壁となっている。札幌市以外の多くの地域では、バス路線の廃止・減便が進み、鉄道も廃線が相次いでいる。このため、自家用車を持たないワーカーは事業所への通勤が困難であり、高齢者の通所サービス送迎や訪問介護の移動時間も長大化している。ワーカーにとって「通勤可能な範囲」が狭く、事業者にとっても「応募可能な人材プール」が限定される悪循環が生じている。

さらに、若年層の道外流出も深刻である。高校・大学卒業を機に札幌圏または道外へ転出する割合が高く、地方部では介護人材の確保がますます困難になっている。これらの複合的な要因により、北海道、特に地方部における介護マッチングは、他都府県と比較して著しく高いハードルを抱えている。

北海道における介護人材確保の特殊事情

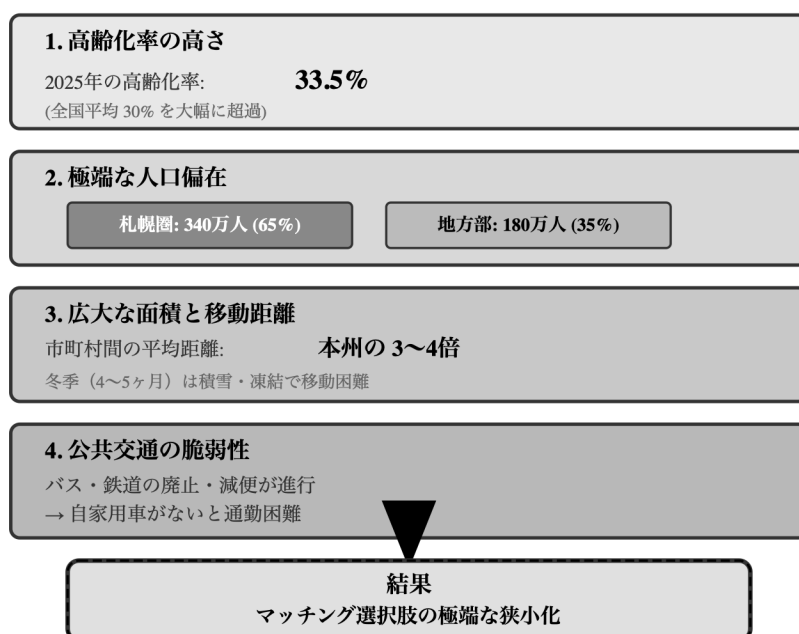


図 1.5: 北海道における介護人材確保の特殊事情

■**現行マッチングシステムの限界** 現状のマッチングシステムは、図 1.6 に示すように「勤務地」「日程」「資格要件」「介護種別」「給与範囲」などの定型条件によるフィルタリングが中心である。具体的には、以下のような検索パラメータでの絞り込みのみが実装されている：

- **地理的条件：** 都道府県 (prefecture)、市区町村 (city) による完全一致フィルタ
- **時間的条件：** 勤務開始日 (start_date)、終了日 (end_date) による範囲検索
- **資格条件：** 介護福祉士、ヘルパー等の資格種別 (qualifications) による完全一致
- **給与条件：** 日給の最低額・最高額 (dailySalaryFrom/To) による範囲検索
- **介護種別：** 訪問介護、施設介護等 (careType) のカテゴリ一致

この手法には以下の構造的な問題が存在する。第一に、**希望職種と求人内容の意味的マッチングが欠如**している。例えば、ワーカーが「身体介護中心の業務」を希望していても、求人票に記載された職務内容のテキスト（「入浴介助」「移乗介助」等）との意味的な一致度は評価されない。第二に、**過去の応募・勤務履歴が活用されていない**。実際のワーカー行動として、良い施設には繰り返し応募する「リピート志向」が観察されるが、このパターンは推薦アルゴリズムに反映されていない。

第三に、**実質的な利便性要因が考慮されない**。ユーザーインタビューから、「駐車場の有無」が重要な選択基準であることが判明した。交通費が往復 500 円の場合、駐車場がなければ地下鉄利用で実質赤字となるケースがあり、名目上の給与だけでは判断できない。しかし現システムには駐車場情報フィールド (has_parking)すら存在しない。第四に、**業務内容の詳細なマッチングが不可能**である。「入浴介助を避けたい」といった具体的な希望があっても、求人検索では「介護種別」という粗い分類でしか絞り込めない。

その結果、ワーカーは毎回多数の検索条件を手動で指定し、表示された求人リストから個別に内容を確認する必要がある。現在の Flutter アプリ実装 (search_screen.dart) では、条件指定のたびに fetchJobs() が呼ばれ、バックエンドへの問い合わせが発生するため、検索体験の負荷が大きい。また、事業者側のスカウト機能も、同様の定型条件（対応可能都道府県 (available_prefecture)、対応可能市区町村 (available_city)、資格 (qualifications) 等）でのフィルタリングにとどまり、ワーカーとの潜在的な親和性は評価できていない。

介護マッチングアプリの仕組み

本研究で対象とする介護特化型マッチングアプリの概要を図 1.7 に示す。このシステムでは、介護事業者が求人情報を登録し、ワーカー（潜在介護士、副業希望者、その他就労希望者等）が希望の勤務地・時間・職種等を入力することで、システムが自動的にマッチングを行う。

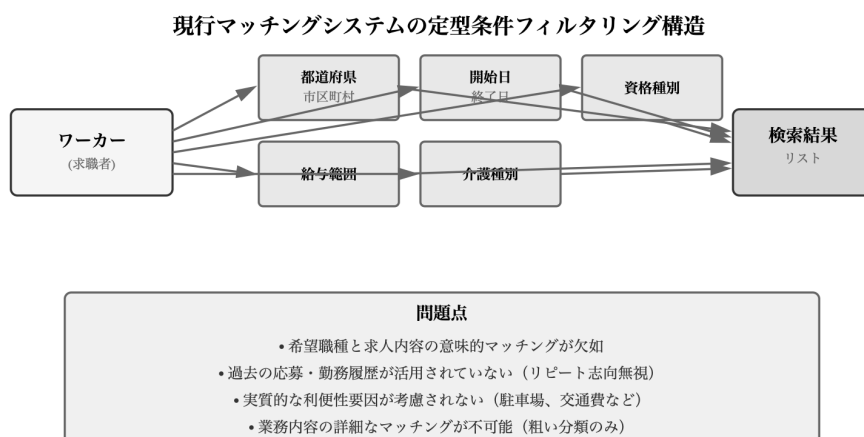


図 1.6: 現行マッチングシステムの定型条件フィルタリング構造

これらの課題を踏まえ、本研究では単純な定型条件マッチ（勤務地・日程・資格等）だけでなく、テキストの意味的類似性や履歴情報、時間的要因を組み合わせた重み付けにより、ワーカーと求人の潜在的親和性をより高精度に評価する手法を提案する。

システムアーキテクチャ概要と設計思想

本研究で構築したシステムは、マイクロサービスの疎結合アーキテクチャを採用し、スケーラビリティと保守性を重視した設計となっている。図 1.8 に全体構成を示す。

設計の主要な特徴として、(1) **関心事の分離**：事業者向け・ワーカー向け・管理者向けの各フロントエンドを独立開発可能とし、異なる技術スタック（Angular/Flutter）を採用できる柔軟性を確保した。(2) **決済処理の分離**：金銭を扱う決済サービスを独立させることで、セキュリティ要件や PCI DSS 準拠を局所化し、他サービスへの影響を最小化した。(3) **バッチ処理の独立性**：日次・月次の定期処理を cron-jobs として分離し、リアル

介護特化型マッチングアプリの概要

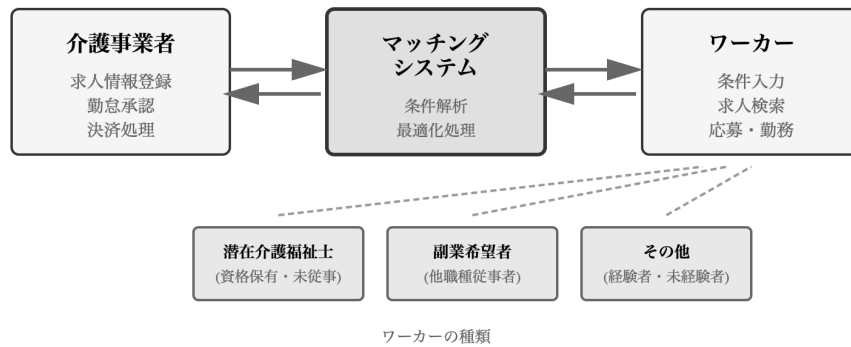


図 1.7: 介護特化型マッチングアプリの概要

タイム API サーバーへの負荷を回避した。(4) 統一的なデータベース：複数サービス間でデータベースを共有することで、データの一貫性を保ちつつ、各サービスは必要なコレクションのみにアクセスする。この構成により、マッチングアルゴリズムの改善や NLP モデルの追加といった機能拡張を、既存サービスへの影響を最小限に抑えながら実施できる。

ビジネスプランと持続可能な価値循環

本システムの事業モデルは、取引成立時の手数料収益を基盤とし、取引量の増加とマッチング精度の向上が相互に強化し合う「データ駆動の改善サイクル」を設計している。図 1.9 に、主要ステークホルダー間の価値の流れと、データ蓄積・学習・改善のフィードバックループを示す。

具体的な価値循環のメカニズムは以下の通りである。(1) 求人掲載から応募へ：事業者

システムアーキテクチャ概要（主要サービス間の疎結合構成）

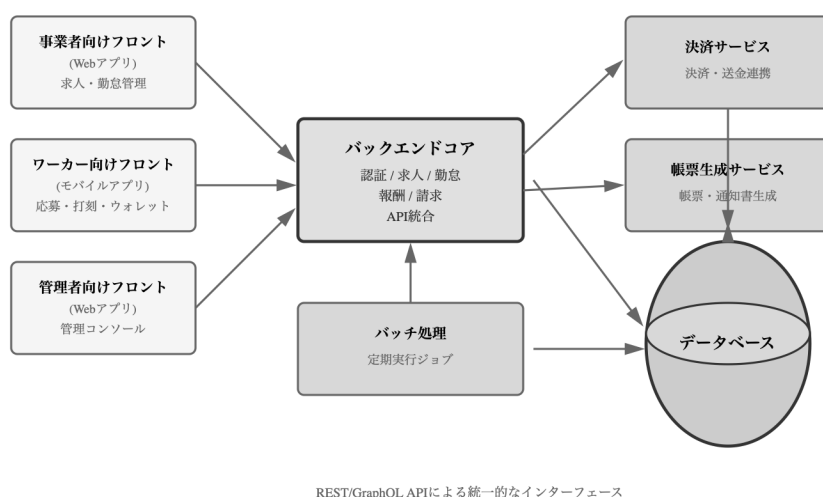


図 1.8: システムアーキテクチャ概要（主要サービス間の疎結合構成）

が掲載した求人情報は、NLP モデルによる意味的類似性計算と過去の履歴分析に基づき、適合度の高いワーカーに推薦される。(2) **応募から勤務実行へ**：ワーカーの応募・採用・勤怠打刻の一連のデータが蓄積され、どの求人条件が実際の就業に結びつきやすいかが分析される。(3) **支払いと手数料回収**：勤務実績に基づく賃金計算と送金が自動化され、プラットフォームは手数料を回収する。この収益が、さらなるアルゴリズム改善やシステム拡張への投資原資となる。(4) **データ学習とモデル更新**：蓄積された応募履歴、勤怠実績、評価データから、マッチング推薦モデルが継続的に学習・更新される。マッチング精度が向上すれば、ワーカーの満足度と定着率が高まり、事業者の採用効率も改善する。その結果、プラットフォームの取引量が増加し、さらに多様なデータが蓄積されるという正のフィードバックが生まれる。このサイクルにより、初期投資を回収しつつ、長期的に持続可能な事業成長が実現される。

ビジネスプランと価値循環（データ駆動の改善サイクル）

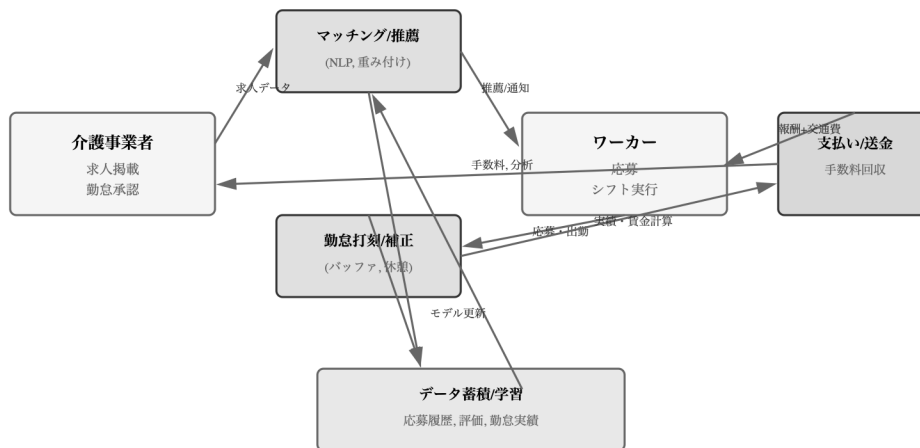


図 1.9: ビジネスプランと価値循環（データ駆動の改善サイクル）

第 2 章

研究の目的

本研究の目的は、現行システムの定型条件フィルタリングの限界を超え、**ワーカーと求人**の「**意味的・潜在的な親和性**」を多角的に捉えることで、双方の検索・推薦の手間を最小化し、マッチング精度を向上させることである。

具体的には、以下の 3 つの側面から改善を図る：

1. **意味的類似性の導入**：自然言語処理（BERT 等）により、ワーカーの希望職務内容と求人の業務記述テキスト間の意味的な距離を計算し、従来の「介護種別」といった粗いカテゴリ分類を超えた精緻なマッチングを実現する。
2. **行動履歴の活用**：ワーカーの過去の応募・勤務履歴から、「リピート施設」への志向性、好む業務内容のパターン、勤務頻度の傾向等を学習し、推薦システムに反映する。これにより、明示的に条件指定しなくても適切な求人を提示できる。
3. **実質的利便性要因の考慮**：駐車場の有無、実質的な交通費負担、移動時間コスト等、名目上の給与や勤務地だけでは捉えられない「実質的な働きやすさ」を定量化し、スコアリングに組み込む。

従来のマッチングシステムの限界

従来のマッチングシステムは、第 1 章で述べたように、都道府県・市区町村・資格・給与範囲・介護種別といった定型条件による完全一致または範囲検索が中心である。実装レベルでは、データベースクエリにおける filter 条件の組み合わせ（都道府県（prefecture）、市区町村（city）、資格（qualifications）、日給（dailySalary）、介護種別（careType）等）で実現されており、これらはすべて構造化データの厳密な一致判定に基づく。このアプローチには以下の問題点がある：

- **条件指定の煩雑さ**：ワーカーは勤務地、時給、勤務時間、業務内容など複数の条件を手動で入力・調整する必要があり、検索コストが高い。
- **意味的ミスマッチ**：「介護」と「看護」、「入浴介助」と「身体介護」など、同義・類

義の表現が統一されていないため、条件が厳密に一致しない求人は検索結果から漏れる。

- **嗜好の変化への非対応**：ワーカーの嗜好は時間とともに変化するが、従来システムは過去の全履歴を均等に扱うため、最新の嗜好が反映されにくい。
- **推薦精度の不足**：事業者側も適切なワーカーを見つけるために手動で履歴を確認する必要があり、マッチング効率が低い。

その結果、潜在的に適合するワーカーと求人が見逃され、双方にとって機会損失が発生している。

本研究のアプローチ

本研究では、これらの課題を解決するために以下のアプローチを採用する：

1. **自然言語処理によるベクトル化**：求人タイトルや業務内容を BERT などの事前学習済み言語モデルでベクトル化し、意味的類似度を計算することで、表現の揺らぎを吸収する。
2. **複数項目の重み付けスコアリング**：求人タイトル、業務内容、介護項目、施設、勤務時間、時給、勤務日など 7 項目を抽出し、各項目に重みを付与したスコアリングモデルを構築する。
3. **過去の応募履歴の活用**：ワーカーの過去の応募パターンから嗜好を学習し、パーソナライズされた推薦を行う。
4. **時間的重み付けの導入**：直近 15～20 件の履歴に高い重みを与え、古いデータの影響を軽減することで、現在の嗜好に適合した推薦を実現する。
5. **計算コストの最適化**：推論コストと精度のバランスを考慮し、リアルタイム性を損なわない推薦方式を設計する。

期待される効果

本研究により、以下の効果が期待される：

- **ワーカー側**：検索条件の入力が不要となり、過去の行動から自動的に適切な求人が推薦される。リピート施設の発見や、新しい施設への応募ハードルが低減する。
- **事業者側**：適切なワーカーへの求人提示が自動化され、採用効率が向上する。ミスマッチによるキャンセルや早期離職が減少する。
- **システム全体**：マッチング精度の向上により、プラットフォーム全体の取引量と満足度が向上する。

これらのアプローチを通じて、介護業界における人材マッチングの効率化と、ワーカー・

事業者双方の満足度向上を目指す。

第 3 章

提案手法

本研究では以下の要素を組み合わせた重み付けを提案する。

自然言語処理モデルを利用した距離計算

求人タイトルや業務内容、職務経歴などのテキストを自然言語処理モデルでベクトル化し、文書間の距離（類似度）を計算する。これにより職種の距離やスキルの距離を定量化できる。

ベクトル化とコサイン類似度

自然言語処理モデル（BERT 等）を用いて、テキストを高次元ベクトル空間に埋め込む。具体的には、求人タイトルや業務内容などの各テキストフィールドをモデルに入力し、固定長のベクトル表現を得る。

テキスト間の類似度評価には、ベクトル化された表現間のコサイン類似度を用いる。2 つのベクトル $\mathbf{a}$ と $\mathbf{b}$ のコサイン類似度は以下で定義される：

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$$

ここで、 $\mathbf{a} = (a_1, a_2, \dots, a_n)$ および $\mathbf{b} = (b_1, b_2, \dots, b_n)$ は n 次元ベクトル、 $\mathbf{a} \cdot \mathbf{b}$ は内積、 $\|\mathbf{a}\|$ および $\|\mathbf{b}\|$ はそれぞれのベクトルのユークリッドノルムを表す。

コサイン類似度は $[-1, 1]$ の範囲の値を取り、1 に近いほど類似度が高く、0 は直交（無関係）、-1 は完全に反対の方向を示す。テキスト解析では通常、非負の値域 $[0, 1]$ で扱われる。

行列間の類似度計算

複数のテキストフィールド（例：求人タイトル、業務内容、介護項目など）を持つ文書全体の類似度を計算する場合、各フィールドをベクトル化した後、それらを行列として扱う。

文書 A のフィールドベクトルを $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_m$ 、文書 B のフィールドベクトルを $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m$ とする。このとき、行列間の類似度は以下の手順で計算される：

1. 各フィールドペア $(\mathbf{a}_i, \mathbf{b}_i)$ のコサイン類似度 s_i を計算
2. 各フィールドに重み w_i を付与し、加重平均を求める：

$$\text{Similarity}(A, B) = \frac{\sum_{i=1}^m w_i \cdot s_i}{\sum_{i=1}^m w_i}$$

この手法により、複数の観点から文書間の類似性を総合的に評価できる。重み w_i は実験的に調整され、本研究では7つの項目（求人タイトル、業務内容、介護項目、施設、勤務時間、時給、勤務日）に対して様々な重み付けパターンを検証した。

時間的距離と勤務条件の差分

勤務地や勤務時間、勤務日などの構造化された条件に基づき時間的・地理的な差分を計算し、ベクトル類似度と組み合わせることで総合スコアを算出する。

過去応募履歴の活用

過去の応募履歴や承認履歴を用いてユーザーの志向性をモデル化する。学習データ（過去 N 件）から平均ベクトルを算出し、新しい求人と類似度を評価する手法を本研究では実験的に採用した。

第 4 章

実験

本節では、本研究で実施した予備実験および本実験の設計と結果を示す。

実験システムの実装とアーキテクチャ

実データ分析の実験環境は `analyze_real_data` ディレクトリに実装し、推薦モデル・評価器・データ処理・Web GUI を分離した多層構成として設計した。研究目的での再現性と拡張性を重視し、CLI/GUI のどちらからでも同一の評価パイプラインを呼び出せるようにした。

技術スタック

- **Python:** 研究実装の中核言語。モデル推論・評価・可視化を一貫して実装。
- **PyTorch + Transformers:** Sentence-BERT によるテキスト埋め込みと類似度計算。
- **FastAPI + Uvicorn:** Web GUI/REST API を提供。非同期処理と並列実行に対応。
- **pandas / numpy / scikit-learn:** データ整形、集計、評価指標の算出。
- **外部データソース (任意):** 実データ取得時の補助的なデータソース (環境変数で有効化)。
- **Jinja2 / Static Files:** GUI テンプレートと静的資産を分離し、軽量な可視化を実現。
- **pymongo / python-dotenv:** 環境変数で本番 DB 連携を制御。

ディレクトリ構成と責務分離

- **app/core/recommenders**: 推薦エンジン本体。Sentence-BERT ベースの推薦と評価クラスを実装。
- **app/core/features**: インタビュー知見に基づく特徴量計算（リピート志向、報酬、業務内容など）。
- **app/analyzers**: 職種のみ・介護項目のみといった軽量ベースラインを実装。
- **app/utlils**: データ読み込み、検証、可視化、設定管理。
- **app/web**: FastAPI による GUI/API 層。テンプレートと静的ファイルを含む。
- **scripts**: 単一分析、バッチ分析、比較実験の実行スクリプト。
- **data/results/tests**: 実データ、出力、テストコードを分離配置。

システムアーキテクチャ（層構造）

以下の層構造により、研究用のオフライン分析と対話的な評価を同一コードベースで実現している。

1. **データ層**: JSON/Markdown 形式のワーカーデータ、求人データを保持。
2. **前処理層**: セッション検証、テキスト正規化、特徴抽出を行う。
3. **モデル層**: Sentence-BERT による埋め込み生成と類似度計算。
4. **推薦ロジック層**: 重み付けパターン、履歴平均ベクトル、ベースライン手法の切替。
5. **評価層**: Top-K、NDCG 等の指標算出と学習曲線評価。
6. **インタフェース層**: CLI 実行スクリプトと FastAPI Web GUI。

アーキテクチャの論理図（文章化）

本システムは、**Web GUI/CLI** を入口として **評価パイプライン** を起動する構成である。GUI 層は FastAPI が担当し、ワーカー選択・実験条件（重み付けや特徴量モード）・進捗表示を提供する。GUI/CLI からの実行指示は評価層に渡され、**データ読み込み**（JSON/外部データ）→**前処理**→**特徴生成**→**推薦**→**評価**→**出力保存** の順に処理される。推薦層は「BERT ベース」「ベースライン」「インタビュー特徴量の統合」を切り替え可能で、同一インタフェース上で比較できる。出力層は JSON/テキスト/HTML に対応し、再現性の高い記録を残す。

データフロー（処理パイプライン）

1. データ読み込み：data_loader.py がワーカー履歴と求人リストを取得。

2. セッション検証：欠損や不正データを除外し、学習用・評価用に分割。
3. 特徴生成：求人テキスト（タイトル・業務内容・介護項目など）を統合しベクトル化。
4. スコアリング：平均ベクトルとのコサイン類似度、または特徴量スコアを計算。
5. 推薦生成：Top-K を抽出し、正解求人の順位を評価。
6. 出力保存：JSON/テキスト/HTML に結果と詳細ログを保存。

データモデル（主要フィールド）

実データはワーカー単位の履歴として保存され、セッションは「応募/勤務1回」を基本単位とする。評価では、各セッションに含まれる**選択求人**と**候補求人集合**を用いる。

- **ワーカー情報**: ワーカー ID (`workerId`)、氏名 (`name`)、資格 (`qualifications`) など。
- **セッション情報**: 勤務日 (`workDate`)、開始時刻 (`startTime`)、職種名 (`jobTitle`)、業務内容 (`jobContent`)。
- **求人情報**: 介護項目 (`careItems`)、施設名 (`facilityName`)、給与 (`salary`)、交通費 (`transportationAllowance`) など。
- **候補集合**: その時点で提示された候補求人群 (`availableJobs`)。

推薦アルゴリズム構成

- **Sentence-BERT ベース**: 求人タイトル・業務内容・介護項目・施設・勤務条件など複数項目の重み付けで統合類似度を算出。
- **重み付けパターン比較**: 20 種類以上の重み付けパターンを並列評価し、最適構成を探索。
- **ベースライン**: 職種のみ／介護項目のみで推薦する軽量モデルを実装し、BERT との差分を測定。
- **インタビュー特徴量**: リピート施設志向、駐車場・交通費、業務内容の詳細一致、報酬の心理的効果、タイミング傾向をスコア化。
- **業務回避パターン**: 過去履歴から避けている業務を推定し、ペナルティを付与。

スコア統合の考え方

BERT スコアは職種や業務内容の**意味的距離**を評価し、インタビュー特徴量は**現場の意思決定要因**（リピート・交通費・業務負荷・報酬印象）を補正する。両者は単純加重和で統合されるが、**利用頻度に応じて重みを動的に調整**し、ライトユーザーとヘビーユーザー

で推薦方針を切り替える設計としている。

評価設計

- **Top-K 精度:** Top-1/3/5 を基本指標とし、実務的推薦の再現性を評価。
- **分割方法:** 適切分割 (学習: テスト = 7:3) とスライディングウィンドウ評価を実装。
- **学習曲線評価:** 学習件数を段階的に増やして性能変化を分析。

評価指標の補足

Top-K 精度に加え、**NDCG** を計算し、順位の質を評価する。Top-K は正解が上位に入るかを測る一方、NDCG は**上位に来るほど高得点**となるため、実サービスでの露出順位を意識した指標として扱う。

高速化・キャッシュ戦略

- **事前エンコード:** 求人埋め込みを事前計算し、重み付け比較時の再計算を回避。
- **キャッシュ層:** メモリキャッシュとディスクキャッシュを併用し、再実験の時間を短縮。
- **並列実行:** FastAPI 上で重み付けパターンを非同期で並列評価。
- **Apple Silicon 最適化:** MPS/スレッド設定を利用し高速化。

障害対策と再実行性

大規模バッチ実験では、途中失敗が研究コストに直結するため、**進捗ログと部分結果の保存**を重視した。GUI からの実行ではキャンセルフラグを監視し、エラー時は原因を JSON に記録して再実験を容易にする。

Web GUI と運用

- **FastAPI GUI:** ブラウザからワーカー選択・分析実行・進捗確認が可能。
- **API 連携:** JSON 形式で分析結果を取得でき、可視化・報告書作成に連携。
- **外部 DB 連携 (任意):** 実データ取得時に最新履歴を取得可能 (環境変数による切替)。

API 設計の要点

GUI と同一のエンドポイントを API として公開し、分析の**自動化**と**外部連携**を可能にしている。単一ワーカー分析、バッチ分析、重み付け比較、事前エンコードといった操作が API 経由で呼び出せるため、研究プロセスの自動化に適する。

出力と再現性

- **結果出力**: JSON/CSV/テキストに加え、HTML レポートとして可視化。
- **ログ管理**: 実行ログ・進捗ログを保存し、追試可能な実験履歴を保持。
- **設定管理**: `config.py` にモデル名や評価パラメータを集約。

拡張性

構成を**推薦エンジンの差し替え**と**特徴量の追加**に耐える設計とした。新規特徴量は `features` に追加し、統合スコアに組み込むだけで再評価できる。新しい推薦モデルも `recommenders` に追加し、同じ評価器に接続できる。

予備実験 1：日本語 BERT モデルによる単語間類似度測定

自然言語処理モデルの代表例として BERT がある。本研究では日本語に特化した `cl-tohoku` モデルを利用し、単語間のコサイン類似度を計測した。しかし、「介護」と「極悪非道」の類似度が 0.678 であったり、「看護師」と「ナース」よりも「介護士」と「ナース」のほうが数値的に近かったりと、直感とずれる結果が得られた。この結果から、単一モデルに依存せず、横断的に複数モデルを検討する重要性が示された。

予備実験 2：BERT モデルの横断評価

複数の BERT 系モデル（日本語 Sentence-BERT、マルチリンガル Sentence-BERT、BERT-large など）を比較し、職種間の分離度を評価した。分離度は「アンカー職種に対する類似職種の平均スコア」と「非類似職種の平均スコア」の差として定義する。

評価データは `reference/類似度検索.py` の設定に合わせ、5 つのアンカー職種ごとに「類似 4 件・非類似 4 件」を手動で作成した。アンカーは店舗業務・IT・営業・医療・データ分析を含むように選定し、語彙の近さだけでなく職域の近さが反映されるかを確認した。

比較対象モデルは、BERT 系の **CLS** 埋め込みと平均プーリング、さらに Sentence-BERT の日本語・多言語モデルである。いずれもコサイン類似度を用い、各アンカーに対

表 4.1: 予備実験 2 の評価データ（アンカーと類似/非類似職種の例）

アンカー	類似職種（例）	非類似職種（例）
レジ打ち	レジ係、キャッシャー	プログラマー、医師
プログラマー	ソフトウェアエンジニア、開発者	販売員、料理人
営業職	セールス、営業担当	経理、研究員
看護師	ナース、病棟看護師	デザイナー、建設作業員
データサイエンティスト	データアナリスト、ML エンジニア	警備員、調理師

して類似/非類似の平均スコア差（分離度）を算出し、アンカー平均をモデルの総合指標とした。

表 4.2: BERT モデル別の分離度スコア

モデル	分離度スコア
Sentence-BERT (Japanese)	0.398
Sentence-BERT (Multilingual)	0.349
BERT-large (Mean Pooling)	0.111
BERT-large (CLS)	0.063
BERT (Mean Pooling)	0.039
BERT (CLS)	0.019

モデル間の違いは大きく二点ある。第一に、Sentence-BERT は文類似度タスクで学習された**文表現**を直接出力するのに対し、通常の BERT は**単語文脈表現**が主であり、文全体の類似度に最適化されていない。第二に、プーリング戦略の違いであり、**CLS** は単一トークンの表現に依存するため短文の意味が安定しにくい一方、**平均プーリング**は全トークンの情報を平均化して文全体の意味を捉えやすい。さらに、BERT-large はパラメータ規模が大きく表現力が高いが、本実験では**日本語特化 Sentence-BERT** が最も高い分離度を示し、**多言語 Sentence-BERT** は言語汎用性の代わりに日本語の精度がやや低下したと解釈できる。

結果から、日本語 Sentence-BERT が最も高い分離度を示し、マルチリンガル Sentence-BERT がそれに続いた。BERT 系では平均プーリングが CLS より安定して高い傾向があり、特に BERT-large の平均プーリングは同系列の中で最良となった。これは単語単位の類似度よりも、短い職種表現全体の意味整合性を捉える埋め込みの方が、職種判別に向いていることを示唆する。

以上の結果を踏まえ、分離度が最も高かった日本語 Sentence-BERT を今後の実験で標準モデルとして採用する。

本実験 1：過去の応募履歴からの推定

学習データとして過去 30 件、テストデータ 10 件を用い、過去応募の平均ベクトルから次に選択される求人を推測した。評価指標として Top-1/Top-3/Top-5 精度および複合スコアを算出した。

表 4.3: 本実験 1：ワーカー別の推薦精度

順位	ワーカー	Top-1	Top-3	Top-5	複合スコア
1	ワーカー A	100.0%	100.0%	100.0%	1.0000
2	ワーカー B	100.0%	100.0%	100.0%	1.0000
3	ワーカー C	60.0%	80.0%	80.0%	0.7000
4	ワーカー D	60.0%	60.0%	60.0%	0.6000
5	ワーカー E	20.0%	80.0%	80.0%	0.5000
6	ワーカー F	10.0%	80.0%	80.0%	0.4500
7	ワーカー G	20.0%	30.0%	40.0%	0.2700
8	ワーカー H	10.0%	30.0%	60.0%	0.2600
9	ワーカー I	0.0%	30.0%	40.0%	0.1700
10	ワーカー J	10.0%	10.0%	30.0%	0.1400
全体平均 (10 名)		39.0%	60.0%	67.0%	0.5090

多く働いているユーザー（ワーカー A、B）は 100% の精度で予測できるが、利用頻度が低いユーザー（ワーカー I、J）は精度が大幅に低下する傾向が見られた。

本実験 2：複数項目の重み付けによる精度向上

求人タイトル、業務内容、介護項目、施設、勤務時間、時給、勤務日など 7 項目を抽出し、各項目に重みを付与して評価した。以下に一部結果を示す。本実験 2 で使用した重み付けパターンは、GUI アプリケーションの選択肢に合わせ、実装側の重み定義（weight_patterns）を参照して整理した。重みの並びは「職種/業務内容/介護項目/施設/勤務時間/時給/勤務日」の順である。

表 4.4: 本実験 2 の重み付けパターン一覧

パターン ID	GUI 表示名	重み	意図
equal	均等	1/1/1/1/1/1/1	全項目を同一比率で扱う基準。
content_focused	業務内容重視	1/3/1/1/1/1/1	業務内容の意味一致を強調。
salary_focused	時給重視	1/1/1/1/1/3/1	報酬条件を優先。
job_title_focused	職種重視	3/1/1/1/1/1/1	職種名の一致を重視。
care_focused	介護・施設重視	1/1/2/2/1/1/1	介護項目と施設種別を優先。
work_time_focused	勤務時間重視	1/1/1/1/3/1/1	勤務時間の条件一致を強調。
extreme_salary	極度時給重視	1/1/1/1/1/5/1	時給のみを極端に強調。
content_salary	業務内容 × 時給	1/3/1/1/1/3/1	業務内容と報酬を同時に強化。
content_salary_strong	業務内容 × 時給強化	1/4/1/1/1/4/1	業務内容と時給をさらに強調。
content_care_salary	業務 × 介護 × 時給	1/3/2/2/1/2/1	仕事内容と介護要素と報酬のバランス。

パターン ID	GUI 表示名	重み	意図
comprehensive	総合重視	1/3/2/2/2/3/1	内容・介護・条件を幅広く強化。
comprehensive_strong	総合重視強化	2/4/2/2/2/4/1	総合重視の強化版。
conditions_focused	勤務条件重視	1/1/1/1/3/3/1	勤務時間と時給に集中。
salary_time_strong	時給 × 勤務時間強化	1/1/1/1/4/4/1	条件重視をさらに強化。
balanced_high	高バランス	2/2/1/1/1/2/1	主要 3 項目を均等に強化。
content_dominant_balanced	業務主導バランス	2/4/1/1/1/2/1	業務内容を軸にバランス配分。
extreme_content	極端業務内容	1/5/1/1/1/1/1	業務内容を大きく強調。
ultra_content	超強力業務内容	1/7/1/1/1/1/1	業務内容への依存度をさらに高める。
mega_content	メガ業務内容	1/10/1/1/1/1/1	業務内容を最重要視する極端設定。
mega_content_salary	業務 × 時給	1/8/1/1/1/5/1	業務内容と時給を極端に強化。
practical_focus	実務重視	1/3/3/3/1/1/1	現場実務に関わる要素を強化。
content_care_strong	業務 × 介護強化	1/4/3/3/1/1/1	業務内容と介護要素を強調。
content_care_avoidance	避ける業務考慮	1/5/5/1/1/1/1	避けたい業務の影響を反映。
care_items_dominant	介護項目極端重視	1/2/8/1/1/1/1	介護項目の有無を最重視。
content_care_balanced	業務 × 介護バランス	1/4/4/2/1/2/1	業務内容と介護項目のバランス。
task_detail_focused	タスク詳細重視	0/6/6/0/0/0/0	業務内容と介護項目のみで評価。
avoidance_optimized	避ける業務最適化	1/4/4/3/1/2/1	業務・介護・施設をバランス強化。
care_items_salary	介護項目 × 時給	1/2/5/1/1/4/1	介護項目と報酬のバランス。
ultra_care_items	超介護項目重視	1/1/10/1/1/1/1	介護項目を最優先に扱う。

表 4.5: 本実験 2 (20 件学習) の全パターン

順位	パターン名	Top-1	Top-3	Top-5	複合スコア
1	均等	57.1%	70.6%	77.1%	0.7112
2	介護・施設重視	58.2%	68.2%	77.6%	0.7094
3	勤務時間重視	52.9%	68.2%	77.1%	0.6959
4	時給 × 勤務時間強化	54.1%	69.4%	75.9%	0.6959
5	時給重視	51.8%	69.4%	76.5%	0.6941
6	勤務条件重視	55.3%	67.7%	75.9%	0.6929
7	極度時給重視	52.9%	65.3%	74.7%	0.6753
8	高バランス型	49.4%	65.3%	74.1%	0.6653
9	業務内容 × 時給	51.8%	62.9%	74.1%	0.6629
10	求人タイトル重視	55.3%	67.1%	70.0%	0.6618

表 4.6: 本実験 2 (30 件学習) の全パターン

順位	パターン名	Top-1	Top-3	Top-5	複合スコア
1	総合重視	30.0%	56.0%	63.0%	0.5430
2	時給 × 勤務時間強化	38.0%	49.0%	63.0%	0.5380
3	業務 × 介護 × 時給	32.0%	52.0%	61.0%	0.5250
4	求人タイトル重視	38.0%	51.0%	59.0%	0.5240
5	実務重視	26.0%	55.0%	60.0%	0.5170
6	業務内容 × 時給	37.0%	50.0%	57.0%	0.5090
7	高バランス型	33.0%	54.0%	56.0%	0.5080
8	介護・施設重視	37.0%	47.0%	57.0%	0.5000
9	超業務内容重視	35.0%	48.0%	57.0%	0.4990
10	均等	34.0%	50.0%	56.0%	0.4980

学習データ数を変化させた場合の精度推移では、学習データが 15 件～20 件の時点で最高値を示し、それを超えると精度が悪化する傾向が観察された。

上記の表から、20 件を超えると全てのパターンで精度が低下することが明確に示された。これは過去の嗜好データが現在の嗜好と乖離するためと考えられる。

表 4.7: 学習データ数による精度推移 (Top-5 精度)

パターン	5 件	10 件	15 件	20 件	25 件	30 件
均等 (equal)	0.69	0.62	0.74	0.77	0.62	0.56
介護・施設重視	0.64	0.64	0.77	0.77	0.61	0.57
時給 × 勤務時間強化	0.68	0.64	0.72	0.76	0.62	0.63
高バランス型	0.70	0.62	0.72	0.74	0.60	0.56
求人タイトル重視	0.68	0.67	0.72	0.70	0.59	0.59

本実験 3：時間重み付け比較

過去の履歴に時間的な重みを付与する複数の減衰関数（指数、パワー、線形、ステップなど）を比較した。表記は「特徴量配分__時間重み」の構成であり、例えば `equal__time_recent_exp_7d` は「特徴量は均等配分 (equal)」「直近 7 日を指数的に重視 (半減期 7 日)」を意味する。対照の `equal__time_equal` は時間減衰を行わず、全履歴を同じ重みで扱う基準である。本実験では特徴量配分 6 種と時間重み 34 種を組み合わせ、合計 204 パターンを評価した。時間重みは日付ベース (`time_recent_...`) とセッション順ベース (`time_session_...`) の 2 系統があり、セッション系は履歴の古さを「直近からの件数」で定義する。

表 4.8: 特徴量配分パターン (本実験 3)

パターン	説明
<code>equal</code>	全特徴を均等配分。
<code>content_focused</code>	業務内容を重視。
<code>salary_focused</code>	時給を重視。
<code>job_title_focused</code>	求人タイトル (職種) を重視。
<code>care_focused</code>	介護項目を重視。
<code>work_time_focused</code>	勤務時間を重視。

2 名データ (学習ステップ 5~40) では、短期リセンシー型 (指数・パワー・反比例・softmax など) が平均 Top-5 で 0.8375 となり、等重み (0.7625) より安定して高い。特徴量配分のカテゴリ別に見ても、`equal` は「全特徴を均等配分」、`content_focused` は「業務内容重視」、`job_title_focused` は「求人タイトル重視」、`care_focused` は「介護項目重視」、`work_time_focused` は「勤務時間重視」を表し、いずれも短期リセンシー型が最良であった。少数ユーザーでは「直近行動の反映」が有効であることが示唆される。

20 名データ (学習ステップ 5~20) では `time_recent_linear_30d` 系 (直近 30 日を線形に重視する時間減衰) が上位を占め、`equal__time_recent_linear_30d` (均等配分 × 30 日線形減衰) と `salary_focused__time_recent_linear_30d` (時給重視 × 30 日線形減衰) が平均 Top-5 で 0.75 となった。等重みの `equal__time_equal` は 0.65 で、時間減衰の設計が精度差に直結している。20 名データには読み込み失敗が 2 件含まれるため、表 4.13 の学習曲線は有効ワーカーの平均に基づく。

表 4.9: 時間重み付けパターン一覧 (1/2)

パターン	意味
time_equal	時間減衰なし (全履歴を同一重み)。
time_recent_exp_7d	指数減衰 (半減期 7 日)。
time_recent_exp_30d	指数減衰 (半減期 30 日)。
time_recent_exp_90d	指数減衰 (半減期 90 日)。
time_session_exp_5	セッション指数減衰 (半減期 5 件)。
time_session_exp_10	セッション指数減衰 (半減期 10 件)。
time_session_exp_20	セッション指数減衰 (半減期 20 件)。
time_recent_linear_30d	線形減衰 (30 日で 0)。
time_recent_linear_90d	線形減衰 (90 日で 0)。
time_session_linear_10	セッション線形減衰 (10 件で 0)。
time_session_linear_20	セッション線形減衰 (20 件で 0)。
time_step_k_5	ステップ関数 (直近 5 件のみ 1、それ以外 0)。
time_step_k_10	ステップ関数 (直近 10 件のみ 1、それ以外 0)。
time_step_k_20	ステップ関数 (直近 20 件のみ 1、それ以外 0)。
time_session_step_k_5	セッション順のステップ関数 (直近 5 件のみ 1、それ以外 0)。
time_session_step_k_10	セッション順のステップ関数 (直近 10 件のみ 1、それ以外 0)。
time_session_step_k_20	セッション順のステップ関数 (直近 20 件のみ 1、それ以外 0)。
time_bi_level_5	2 段階重み (直近 5 件=1、以降=0.2)。
time_bi_level_10	2 段階重み (直近 10 件=1、以降=0.2)。
time_bi_level_20	2 段階重み (直近 20 件=1、以降=0.2)。
time_session_bi_level_5	セッション順の 2 段階重み (直近 5 件=1、以降=0.2)。
time_session_bi_level_10	セッション順の 2 段階重み (直近 10 件=1、以降=0.2)。
time_session_bi_level_20	セッション順の 2 段階重み (直近 20 件=1、以降=0.2)。

表 4.10: 時間重み付けパターン一覧 (2/2)

パターン	意味
time_window_uniform_30d	30 日以内のみ 1、それ以前は 0。
time_window_uniform_90d	90 日以内のみ 1、それ以前は 0。
time_recent_power_1	パワー減衰 ($1/(age + 1)$)。
time_recent_power_2	パワー減衰 ($1/(age + 1)^2$)。
time_recent_log	対数減衰 ($1/\log(age + e)$)。
time_inverse_age_1	反比例減衰 ($1/(age + 1)$)。
time_sigmoid_30d_10d	シグモイド減衰 (中心 30 日、幅 10 日)。
time_decay_then_flat_30d	30 日まででは指数減衰、それ以降は 0.2 固定。
time_softmax_30d	Softmax 減衰 (温度 30 日)。
time_softmax_7d	Softmax 減衰 (温度 7 日)。
time_session_power_1	セッションパワー減衰 ($1/(k + 1)$)。

表 4.11: 時間重み付け比較 (平均 Top-5)

対象	代表パターン	平均 Top-5	備考
2 名	equal__time_recent_exp_7d	0.8375	等重みは 0.7625
20 名	equal__time_recent_linear_30d	0.75	等重みは 0.65

表 4.12: 時間重み付け上位パターン (平均 Top-5)

対象	パターン	平均 Top-1	平均 Top-3	平均 Top-5
2 名	equal__time_recent_exp_7d	0.60	0.80	0.8375
2 名	equal__time_recent_power_1	0.525	0.8125	0.8375
2 名	equal__time_inverse_age_1	0.525	0.8125	0.8375
20 名	equal__time_recent_linear_30d	0.375	0.525	0.75
20 名	salary_focused__time_recent_linear_30d	0.40	0.575	0.75
20 名	content_focused__time_auto	0.35	0.475	0.675

表 4.13: ステップ別平均 Top-5 (有効ワーカー平均)

学習ステップ	2 名 (平均 Top-5)	20 名 (平均 Top-5)
5	0.50	0.70
10	0.5711	0.50
15	0.8005	0.90
20	0.8176	0.6111
30	0.90	—
35	0.9789	—
40	0.8863	—

本実験 4：時間重み自動選択の実装と結果

時間重み付け比較の知見を踏まえ、履歴のドリフト傾向に応じて短期/長期の減衰関数を自動選択する `time_auto` を実装した。GUI では「時間的重み付けパターン」に `time_auto` (自動判別) を用意しており、単一分析・バッチ分析・時間重み比較のいずれも同一パラメータで実行される。実装側では履歴に日付情報が含まれる場合は日付ベースの減衰 (例: `time_recent_exp_7d`) を、日付が取得できない場合はセッション順ベースの減衰 (例: `time_session_exp_5`) を初期候補として選ぶ。

自動選択の中核は「履歴のドリフト」検出である。過去履歴の埋め込みベクトルを作成し、直近ウィンドウ (最小 2 件・最大 5 件、履歴長の 1/3 を上限) とそれ以前の履歴を分け、各集合の平均ベクトル間のコサイン類似度を計算する。ここで $\text{drift} = 1 - \text{similarity}$ と定義し、値が大きいほど最近の行動が過去と異なる (志向性が変化している) と解釈する。ドリフトが小さい場合 (閾値 0.12 未満) は安定とみなし、長期履歴を厚めに使う減衰 (`time_window_uniform_90d` または `time_session_linear_20`) へ切り替える。ドリフトが大きい場合は短期変化の追従を優先し、指数減衰 (`time_recent_exp_7d` / `time_session_exp_5`) を維持する。判定結果 (選択パターン、ドリフト値、最近ウィンドウ長) はワーカー単位にキャッシュし、同一履歴長での再計算を省略する。

`time_auto` の検証は別途 5 名データ (学習ステップ 5~40) で実施し、`equal` 系の時間重みを比較したところ、`equal__time_auto` は等重み `equal__time_equal` を上回り、明示的な線形減衰 `equal__time_recent_linear_30d` が最も高い平均 Top-5 を示した。自動選択は「短期の行動変化」への追従性を高める一方、ドリフト推定は履歴量や埋め込みのばらつきに影響されるため、固定減衰との併用が有効である。

表 4.14: `time_auto` 実装結果 (5 名, 平均 Top-K)

パターン	平均 Top-1	平均 Top-3	平均 Top-5
<code>equal__time_equal</code>	0.5125	0.75	0.7625
<code>equal__time_auto</code>	0.5375	0.7375	0.775
<code>equal__time_recent_linear_30d</code>	0.625	0.80	0.825

第5章

考察

本研究の実験結果から導かれる主要な考察を以下に示す。

- **利用頻度による精度差：**多く働いているユーザー（データが豊富なユーザー）とそうでないユーザーで応募傾向が大きく異なる。頻繁に利用するユーザー（ワーカー A、B）は 100% の精度で予測できるのに対し、利用頻度が低いユーザー（ワーカー I、J）では複合スコアが 0.14～0.17 と大幅に低下する。これはユーザーの利用頻度に応じた重み付け調整の必要性を示唆する。
- **最適学習データ量：**学習データ量については、15 件～20 件程度が最適帯であることが判明した。20 件学習時の最高複合スコアは 0.7112（均等パターン）であるのに対し、30 件学習時は 0.5430（総合重視パターン）まで低下する。これは約 24% の精度低下に相当する。過度に過去のデータを学習すると、古い嗜好が現在の嗜好を汚染し精度が低下するためと考えられる。したがって、時間的な重み付け（古いデータを低減する等）を導入することが有効である。
- **重み付けパターンの選択：**20 件学習では「均等 (equal)」パターンが最良であったが、30 件学習では「総合重視 (comprehensive)」が最良となる。これは学習データ量によって最適な重み付けが変化することを示しており、動的な重み調整が望ましい。
- **時間重み付けの有効性（実験 3）：**時間重み付けでは、等重みよりも直近重視の減衰が上位に現れ、特に線形減衰（30 日で 0）や指数・パワー減衰が安定して高い Top-5 を示した。これは「古い履歴の影響を下げる」ことが精度に寄与することを示唆する。一方で、日付ベース/セッションベースのどちらが優位かはデータ特性に依存し、短期の行動変化が強いワーカーでは急峻な減衰が有効、長期で安定した志向では緩やかな減衰が適する可能性がある。
- **時間重み自動選択の位置づけ（実験 4）：**自動判別は等重みを上回る結果を示し、ドリフトが大きいワーカーに対する追従性を高める効果が確認された。しかし、明示的な線形減衰（30 日）が最良であった点から、自動選択は万能ではなく、履歴量・

埋め込みのばらつき・日付欠損などにより判定が不安定になり得る。したがって、自動選択を初期値や補助として用い、固定減衰との併用・比較を前提とする運用が現実的である。

- **リピート行動の重要性：**ユーザースタディから、ワーカーは一度良いと感じた施設に繰り返し応募する傾向が非常に強いことが判明した。「いつもの施設」が第一選択肢となり、新規施設の探索は限定的である。これは本実験 1 で観察された「多く働いているユーザーは特定の求人に繰り返し応募する」という傾向と一致しており、過去の応募履歴による推薦の有効性を裏付ける。
- **非テキスト要因の影響：**ユーザースタディでは、駐車場の有無や特定業務（入浴介助等）の可否が、時給以上に意思決定に影響することが示された。これらの構造化データは、本研究で提案した 7 項目の重み付けにおいて「勤務条件」として扱われているが、さらに細分化して高い重みを与える必要がある。特に「駐車場」は往復交通費が赤字になる閾値（500 円）との関係で重視されており、経済的合理性に基づく意思決定が行われている。
- **総額志向と表示方法：**給与については時給換算よりも「1 回でがっつり稼ぎたい」という総額志向が見られた。これは UI において時給だけでなく、想定総収入を明示することの重要性を示唆する。
- **口コミと横のつながり：**知人からの情報共有が活発であり、「ここはヤバかった」といった具体的な評判が意思決定に強く影響する。これは本研究のスコアリングモデルには含まれていない要素であり、今後は評価スコアやレビューデータを統合することが課題である。
- **フィードバック機能の心理的效果：**レビュー機能や QR コード読み取り時のメッセージは、単なる確認手段以上の価値を持つ。「役に立てた実感」というワーカーの内発的動機付けを高め、サービス継続利用につながる可能性が示された。
- **オンボーディングの情報非対称性：**単発マッチングにおける最大の課題は、初回訪問時の情報格差である。動画解説や事業所紹介記事など、事前情報提供の充実が応募ハードルを下げ、マッチング成功率を向上させる可能性がある。
- **マルチチャネル対応の二重性：**アプリと LINE の両方で連絡する二度手間が指摘された一方で、LINE サポートは高く評価されている。この矛盾は、チャネル統合ではなく、各チャネルの役割を明確化する設計が重要であることを示唆している。

提案：ユーザーの利用頻度に基づく重み付け、時間的重み付け（直近 15～20 件を重視、線形/指数減衰の併用）、および自動判別の併用を検討することが望ましい。さらに、ユーザースタディで得られた知見を反映して駐車場の有無や特定業務の可否に対して高い重みを設定し、総収入の明示や口コミスコアの統合を行うべきである。加えて、オンボーディングの充実（動画・記事）、フィードバック機能の継続的改善、マルチチャネルの役割明確化が、ユーザーエクスペリエンス向上の鍵となる。

第 6 章

ユーザースタディ

本研究では候補者の選定とインタビューを実施し、ユーザースタディから得られた示唆を整理した。候補者はカスタマーサービスと連携して 13 名を選定し、アンケートとインタビュー項目を用意した。現段階では 30 分程度のインタビューを 2 件実施した。

インタビュー対象者

1 件目のインタビュー対象者は、介護経験 7 年目のワーカーで、現在は副業としてマッチングサービスを月 4～5 回程度利用している。過去に 4 つの事業所での勤務経験があり、サービスの活用方法について豊富な知見を持つ。

2 件目のインタビュー対象者は、実務者研修時に日本福祉アカデミーでモンスケを知り、母親の利用をきっかけに単発バイトとして利用を開始した。複数のサービス機能を実際に利用しており、システムの使い勝手について具体的なフィードバックを提供した。

求人選択の実態

インタビューから、ワーカーの求人選択プロセスには明確な優先順位があることが判明した。

■**リピート志向の強さ** ワーカーは一度良いと感じた施設には繰り返し応募する傾向が非常に強い。「いつも行くところがあれば、そこを選ぶ」という発言から、リピート施設が第一選択肢となっており、新規施設の探索は「いつもの施設が空いていない場合」に限られることが分かった。この背景には、「お互いゼロから教えなくていい」という相互の効率性への配慮がある。

■**駐車場の重要性** 予想外に高い重要度を示したのが「駐車場の有無」である。交通費が往復 500 円で、地下鉄の往復料金を超えると赤字になるため、駐車場がない施設は敬遠される。これは時給以上に意思決定に影響する要因であることが示された。

■**業務内容の選好** 特定の業務（入浴介助等）の有無は、時給よりも優先される場合がある。「入浴がない」ことが施設選択の重要な条件となっており、業務内容の詳細表示が求人の魅力度に直結することが確認された。

■**給与の基準** 給与については「8 時間勤務、1 時間休憩、交通費込みで 1 万円程度」が望ましい水準として挙げられた。時給換算よりも「1 回でがつつり稼ぎたい」という総額志向が見られた。

情報収集の実態

新規施設を選ぶ際、ワーカーは以下の手順で情報を収集していた：

1. 近い場所から順に閲覧
2. 業務内容と口コミを確認
3. 知人からの横のつながりで得た情報を参照

特に「モンスケをやっている友達」からの情報共有が活発であり、「ここはヤバかった」といった具体的な評判が意思決定に強く影響することが判明した。

ユーザースタディで得られた「避けたい業務の存在」を実装に反映するため、次章では回避業務ペナルティの実験とその結果を詳細に整理する。

第 7 章

回避業務ペナルティと時間志向の実験

ユーザースタディで確認された「入浴介助など特定業務を避けたい」という傾向を推薦に反映するため、回避業務にペナルティを適用する実装方針の検証を行った。本章では、追加実験 1 として回避業務ペナルティの効果を整理し、追加実験 2 として時間志向（長期/短期）を同時に扱った場合の影響を確認する。

追加実験 1：回避業務ペナルティの検証

実装方針と GUI 連携

GUI には「業務回避ペナルティを有効化」のチェックボックスを追加し、オンの場合は `enable_task_avoidance=true` を解析 API へ送信する。評価方法の一覧には「業務回避ペナルティ分析」を独立項目として設け、ペナルティ無し/有りを同条件で実行・比較できるようにした。ペナルティ有無の両方を計算するため、実行時間は増加する。

回避業務の検出

各セッションで提示された候補求人 (`availableJobs`) に含まれる介護項目を集計し、実際に選ばれた求人 (`completedJob`) との選択率を算出する。選択率が 30% 未満の業務項目を「回避項目」と判定し、回避度合いは $1 - \text{選択率}$ として保存する。

ペナルティ係数の計算

回避項目が含まれる求人に対して、回避率と回避スコアに基づく減点を適用する。ペナルティ重みは 0.5 とし、以下の式で係数を計算する。

$$\text{penalty_factor} = (1 - r \cdot w) \times (1 - s_{\max} \cdot w)$$

ここで r は求人内の回避項目の比率、 $s_{\max}$ は回避項目の最大回避スコア、 w はペナルティ重みである。推薦スコアにこの係数を乗算し、再ランキングする。

評価設計

重み付けパターン比較をベースに、**ペナルティ無し**と**ペナルティ有り**を同一条件で評価する。出力は Top-1/Top-3/Top-5 の精度比較と回避項目の一覧を提示し、回避傾向が強いワーカーほど順位変化が大きいかを確認する。

結果

20 名データのうち、時間重み `time_auto` の比較結果が揃った 18 名を対象に平均精度を算出した。ペナルティ有りは Top-1/Top-3/Top-5 の全てで改善し、回避傾向が推薦結果に反映されることが確認できた。

順位	パターン	ステップ	Top-5 (無)	Top-5 (有)	差分	Top-5 (time_auto)	差分	Top-5 (有+time_auto)	差分
1	職種重視	40	51.2%	68.8%	+17.50%	57.5%	+6.25%	68.8%	+17.50%
2	勤務時間重視	40	52.5%	68.8%	+16.25%	61.3%	+8.75%	68.8%	+16.25%
3	時給重視	40	55.0%	67.5%	+12.50%	65.0%	+10.00%	68.8%	+13.75%
4	介護・施設重視	40	50.0%	66.3%	+16.25%	60.0%	+10.00%	66.3%	+16.25%
5	均等	40	55.0%	66.3%	+11.25%	65.0%	+10.00%	68.8%	+13.75%
6	業務内容重視	40	56.3%	66.3%	+10.00%	62.5%	+6.25%	67.5%	+11.25%

図 7.1: 20 名データの重み付けパターン比較（概観）

表 7.1: 回避ペナルティ有無の平均精度（18 名, equal / time_auto）

条件	Top-1	Top-3	Top-5
ペナルティ無し	0.487	0.598	0.660
ペナルティ有り	0.542	0.690	0.763
差分	+0.055	+0.092	+0.103

考察

回避業務ペナルティは、ユーザースタディで確認された「避けたい業務」を明示的に減点することで、上位候補の順位を現場の意思決定要因へ寄せる効果がある。特に Top-3/Top-5 での改善が大きく、候補群の質を底上げしていることが示唆される。一方で、回避傾向が弱いワーカーでは差分が小さく、ペナルティ強度の個別調整や回避項目の更新頻度が今後の課題となる。

追加実験 2：時間志向を同時に扱った場合

実装方針と GUI 連携

GUI では「ペナルティ有無 + time_auto 比較」を用意し、ペナルティ無し/有りに加えて time_auto とペナルティ有り + time_auto を同時に評価できる。

長期志向/短期志向の判定

結果表示には時間パターンの自動判別サマリを含め、長期志向 (stable) と短期志向 (recent) をラベルとして表示する。判定には履歴のドリフト、最近ウィンドウ長、日付ベース/セッションベースの別を用い、採用された時間パターンを明示する。

評価視点

回避ペナルティが有効であることを確認した上で、時間志向 (長期/短期) の自動判別を同時に適用した場合の影響を確認する。

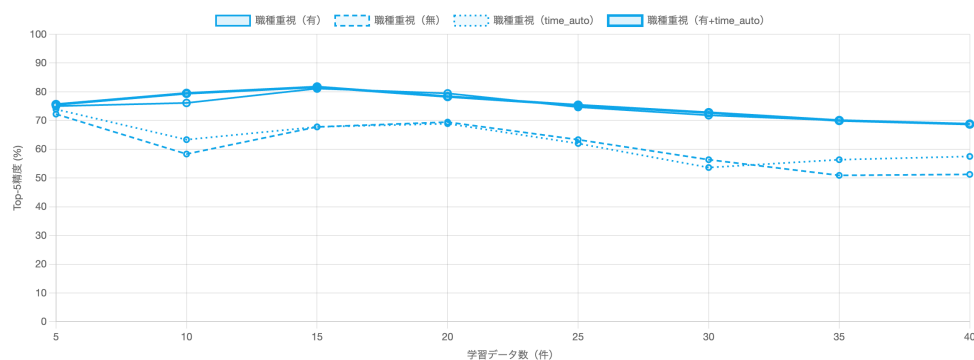


図 7.2: ペナルティ有無 + time_auto の同時評価結果

第 8 章

今後の展望

今後の研究・実装に関する方向性を以下に示す。

1. 本実験 2 で示唆された重み付けの更なる最適化：ユーザーの利用頻度や応募の時間的な重みを組み込み、学習データの選択基準を改善する。
2. 実験 3 の比較検討の拡充：より多くの日本語モデルを横断的に評価し、実運用に適したモデル選定基準を整備する。
3. 実務実装に向けたインタビューと A/B テスト：実際のサービスに実装し、ユーザー行動の変化を観察し重み付けサイクルを改善する。
4. 実験 2,3 の手法を統合した高精度マッチングアルゴリズムの開発：複数の類似度評価を総合してランキングを生成するフレームワークの確立。

第 9 章

結論

結論

本研究では、介護マッチングにおける意味的類似性の導入と複数項目を組み合わせた重み付けが、実務的に有用なマッチング精度を達成し得ることを示した。予備実験では日本語 Sentence-BERT が高い分離度を示し、実験全体の標準モデルとして有効であることが確認できた。

実験では学習データ量や重み付けパターンにより精度が変動し、特に 15 件～20 件程度の学習データで良好な結果が得られた。これは過去データの扱い（時間的重み付け）の重要性を示唆する。

本手法により、ワーカーの希望とより適合した求人を効率的に提示できるシステムの基盤が構築された。今後は、実データを用いたさらなる検証と、複数の手法を組み合わせたより高精度なマッチングアルゴリズムの開発、実サービスへの実装とユーザーフィードバックに基づく改善サイクルの確立が課題である。

システム構成概要（ディレクトリベース）

本研究で取り扱ったプロダクト群のコードベース構成を、リポジトリ直下の主要ディレクトリ単位で整理する。

- **care-provider-fe**: Angular を用いた介護事業者向けフロントエンド。求人作成・勤怠確認・設定管理などを担当。
- **care-worker-fe**: Flutter 製のワーカー向けクライアント。求人検索・応募・勤怠打刻・ウォレット閲覧を提供。
- **kaigo-admin-fe**: 管理者向け Angular フロントエンド。事業所/求人/ユーザ管理や設定変更を行う。
- **kaigo-be**: Node.js/TypeScript ベースのバックエンド API。src/controllers, services, models を中心に、認証、求人・応募管理、勤怠計算、支払い処理などを実装。
- **kaigo-sevenpay-be**: 決済・外部送金系のバックエンド (Node.js/TypeScript)。七銀連携や決済ロジックを分離。
- **pdf-generator**: PDF 生成用の専用サービス (Node.js)。通知書や帳票を生成。
- **cron-jobs**: シェルスクリプトによる定期実行タスク群 (例: 日次・月次バッチ)。
- **documents**: 設計メモ・ドキュメント類 (システム構成、課題整理など)。
- **analyze_real_data**: 実データ分析・推薦評価の実験環境。Sentence-BERT を用いた推薦エンジンと評価器、職種ベース/介護項目ベースの軽量ベースライン、インタビュー知見を統合したハイブリッド推薦、FastAPI による Web GUI、単一分析/バッチ分析スクリプト、可視化・結果出力を実装。
- **master_thesis**: 本論文のソース一式 (L^AT_EX)。

補足: 各バックエンドでは Mongoose を用いたスキーマ定義を採用し、/src/models 配下でドメイン別にモデルを分割している。フロントエンドは Angular/Flutter で機能別にモジュール化されている。

謝辞

本研究および本論文の作成に関し、多大なる御指導、御討論を頂きました北海道大学大学院情報科学研究院メディアネットワーク部門、土橋宜典教授に心より感謝いたします。

本研究に対して、御助言、御協力を頂いた北海道大学大学院情報科学研究科メディアネットワーク専攻情報メディア学講座情報メディア環境学研究室諸氏に心より御礼を申し上げます。

参考文献

- [1] 厚生労働省, ” 介護人材確保の現状について” 厚生労働省 (令和 7 年 5 月 9 日).
<https://www.mhlw.go.jp/content/12000000/001485589.pdf>
- [2] 厚生労働省, ” 福祉・介護人材確保対策について” 厚生労働省 (n.d.).
<https://www.mhlw.go.jp/seisaku/09.html>
- [3] 厚生労働省, ” 福祉人材確保対策” 厚生労働省 (n.d.).
https://www.mhlw.go.jp/stf/seisakunitsuite/bunya/hukushi_kaigo/seikat-suhogo/fukusijinzhai/index.html
- [4] 厚生労働省, ” 介護福祉士の資格等取得者の届出制度” 厚生労働省 (n.d.).
<https://www.mhlw.go.jp/stf/seisakunitsuite/bunya/0000158837.html>
- [5] 厚生労働省, ” 介護人材確保に向けた取組について” 厚生労働省 (n.d.).
https://www.mhlw.go.jp/stf/newpage_02977.html

研究業績

- [1] 星子 祐哉, 青木 直史, 土橋 宜典 “VGG16 の転移学習を利用した病的音声の検出”, 電気・情報関係学会北海道支部連合大会, オンライン開催, November 5-6, 2022.
- [2] 介護マッチングサービスにおけるマッチング精度向上の提案と実装 (星子 祐哉, 土橋 宜典), 令和 7 年度 電気・情報関係学会北海道支部連合大会 (2025/11/2 - 3).